RAG EXPLAINED by Jyothi Prakash Kalmanje V2

Production-Grade Retrieval-Augmented Generation
Architecture · retrieval quality · evaluation · security · operations

1 · WHAT RAG IS, AND WHY IT EXISTS

start here — the problem before the architecture

The definition

Retrieval-Augmented Generation — before answering, the system **searches your own documents**, puts the passages it finds into the prompt, and asks the model to answer **from that evidence** rather than from memory.

One sentence: RAG turns a closed-book exam into an open-book one. The model stops recalling and starts reading.

The four steps

1	**Index** — split your documents into passages, embed them, store them in a searchable index. Done once, ahead of time.
2	**Retrieve** — at question time, find the passages most likely to contain the answer.
3	**Augment** — place those passages into the prompt alongside the question.
4	**Generate** — the model answers using the supplied text, and cites which passage it used.

Concretely

```
// question the model was never trained on
Q: "How many annual leave days do I carry over?"

// 1–2 · retrieve from the HR handbook index
→ hr-policy-2026.pdf p.14, chunk 3  (score 0.89)
→ leave-faq.md  section "Carry-over"    (score 0.81)

// 3 · augment
PROMPT: "Answer using only the context below.
         Cite the source. If the context does not
         contain the answer, say you do not know.
         CONTEXT: <the two passages>
         QUESTION: How many days do I carry over?"

// 4 · generate
A: "Up to 5 days, and they expire on 31 March.
   [hr-policy-2026.pdf, p.14]"
```

The problem it solves

A language model knows only what was in its training data, frozen at a point in time. That leaves five gaps, and RAG addresses each one directly.

What a model alone cannot do	What retrieval changes
Know anything after its training cutoff	The index is updated whenever the source changes — no retraining involved
Know your private or internal data	Your contracts, tickets, wiki and code were never public, and never will be in a public model
Say where an answer came from	The retrieved passage *is* the citation, so the claim can be checked
Reliably admit it does not know	With nothing retrieved, the system can refuse instead of inventing a plausible answer
Respect who may see what	Retrieval is filtered by the user's permissions before anything reaches the prompt

The honest framing. RAG does not make a model truthful. It makes the model's answer **traceable to a source you control** — which is what lets you detect when it is wrong, and is why regulated and internal-knowledge use cases adopted it first.

What it is not

- **Not a hallucination cure.** A model can still misread or over-generalise a correct passage.
- **Not search.** Search returns documents to a human; RAG returns an answer built from them.
- **Not a database.** It finds relevant text, not exact rows — do not ask it to sum a column.
- **Not automatic.** Answer quality is set by retrieval quality, and retrieval is the hard part.

When it is the right tool

Signal	Why RAG fits
Corpus changes	Policies, prices, documentation, tickets — anything you would otherwise retrain for
Answers must be cited	Support, legal, compliance, medical, finance
Data is private	Internal knowledge that cannot leave your boundary
Per-user visibility	One model, retrieval filtered per user or tenant
Corpus is large	Far more text than any prompt could hold
Answers are extractive	The answer exists in a document, and needs finding rather than reasoning out

When it is the wrong tool

Situation	Use instead
Aggregation and maths	"Total revenue by region" is SQL, not retrieval
Small stable corpus	If it fits in the prompt and rarely changes, just include it
Behaviour, tone, format	Fine-tuning — see panel 3
Real-time transactional state	Call the API; an index is always slightly stale
Multi-hop reasoning	An agent that retrieves repeatedly, not one retrieval pass

The first question to ask. Where does the answer live today? If a person would find it by opening a document, RAG fits. If they would run a query, write a report, or make a judgement call, it does not — and no amount of retrieval tuning will change that.

2 · THE PRODUCTION RAG PIPELINE

every stage that a demo skips

A · INDEXING PATH — offline, incremental, must handle updates and deletes

Source systems — CMS · wiki · S3 · DB · tickets · contracts
→ **Parse + extract** — layout, tables, OCR · strip boilerplate
→ **Chunk** — + carry section path + parent linkage
→ **Enrich** — ACL, tenant, date · source, doc version
→ **Embed** — batched, retried · model version pinned
→ **Vector index** — dense vectors + payload · HNSW / IVF
→ **Lexical index** — BM25 / sparse · for exact-token recall

Upsert by stable document ID — Re-ingest replaces prior chunks for that doc. Deleted source ⇒ chunks must be deleted too.

The delete path is the one teams forget — A revoked or retracted document that stays in the index keeps being retrieved and cited. That is a compliance defect.

Both indexes are built from the same chunks. Dense captures meaning; sparse captures the literal token. Production retrieval queries both.
Metadata written here (tenant, ACL, date, version) is what makes filtering, security and freshness possible later — it cannot be retrofitted at query time.

B · SERVING PATH — online, on the critical latency path

Query — + user identity + conversation state
→ **Rewrite / route** — resolve pronouns · decide: retrieve or not
→ **Cache check** — exact + semantic · hit ⇒ skip the rest
→ **Dense search (ANN)** — top ~50–100 candidates
→ **Lexical search (BM25)** — top ~50–100 candidates
→ **Fuse (RRF)** — merge both lists · rank-based, no tuning
→ **Rerank** — cross-encoder · → top 3–10
→ **Assemble context** — expand to parents · order by position bias
→ **LLM** — grounded prompt · cite-or-refuse
→ **Guardrails** — citation check, PII · injection detection
→ **Answer** — streamed · with citations

Trace + feedback loop — Log the query, the candidates, the reranked set, what was sent, what was cited, and the user's verdict. Without this you cannot debug or improve anything.

The four stages a prototype does not have

1 · **Lexical retrieval alongside dense** — recovers exact tokens that embeddings blur.
2 · **Reranking** — commonly the single highest-return upgrade to a naive pipeline.
3 · **Guardrails** — retrieved text is untrusted input, and must be treated as such.
4 · **Tracing** — the difference between engineering the system and guessing at it.

Recall first, then precision — why two stages beat one

Vector search is a **recall** instrument: cheap, approximate, run over millions of chunks. Ask it for the top 3 and the right passage is often ranked 7th and silently dropped. Ask for 50–100 and it is almost certainly in there.
The reranker is a **precision** instrument: expensive per pair, but only ever sees those 50–100 candidates.
Cast a wide net cheaply, then sort it accurately. Neither stage can do both jobs well alone.

WHAT ACTUALLY SEPARATES A DEMO FROM A PRODUCTION SYSTEM

Demo	Production
One embedding model, one vector store, top-k = 3	Hybrid retrieval, reranking, parent expansion, tuned top-k
Judged by "the answers look good"	Judged against a golden set, on retrieval and generation separately
Every user sees every document	ACL enforced *inside* the retrieval filter, per tenant and per user
Index built once, by hand	Incremental indexing with updates, deletes and backfill
Failures are debugged by re-reading the prompt	Failures are triaged with traces, in a fixed diagnostic order
Cost and latency discovered in production	Latency and cost budgeted per stage and monitored

3 · FUNDAMENTALS

what is actually happening

RAG gives a language model access to information it was never trained on. Instead of answering from parameters alone, the system **retrieves relevant source text at query time** and puts it in the prompt. The model then reasons over supplied evidence rather than recalled facts.

Why not just fine-tune?

Need	Right tool
Facts that change	**RAG** — update the index, no retraining
Per-user / per-tenant data	**RAG** — one model, filtered retrieval
Citations required	**RAG** — the source is known by construction
Behaviour, tone, output format	**Fine-tuning** — teaches style, not facts
Domain vocabulary in embeddings	**Embedding fine-tune**

They are complementary, not competing. Fine-tuning changes how a model behaves; retrieval changes what it knows. Mature systems frequently use both.

Why bigger context windows did not remove the need

- **Cost** — input tokens are charged on every call; retrieval sends thousands, not millions
- **Latency** — prompt processing grows with context length
- **Accuracy** — relevant facts buried among irrelevant ones are used less reliably
- **Access control** — you cannot enforce per-user permissions by pasting the whole corpus

Position bias is real and measurable. Published work on long-context behaviour ("lost in the middle", Liu et al., 2023) found models use information at the **start and end** of a context more reliably than material buried in the middle. Practical consequence: after reranking, place the strongest passages at the edges of the assembled context, not in the centre.

The two independent failure surfaces

Failure	Meaning
Retrieval failure	The evidence never reached the model. No prompt can fix this.
Generation failure	Evidence was present and the model still answered wrongly.

Measure them separately. A single end-to-end quality score cannot tell you which half is broken, so it cannot tell you what to fix.

4 · CHUNKING

decided before everything else

Strategy	When it is the right choice
Fixed-size	Prototypes only. Splits mid-sentence and destroys meaning at every boundary.
Recursive separator	Solid general default: split on paragraph, then sentence, then word.
Semantic	Boundaries placed where the topic shifts. Good for continuous prose.
Document-aware	Best for structured content — headings and sections are free, author-supplied boundaries.
Hierarchical (small-to-big)	Search a small precise child, send its larger parent to the model. Precision *and* context.

Parameters that matter TUNE, DON'T COPY

Parameter	Trade-off it controls
Chunk size	Small = precise retrieval, thin context. Large = rich context, diluted embedding.
Overlap	Insurance against boundary loss; costs storage and creates near-duplicates.
Section path in text	Prefixing "Returns › Damaged items" to a chunk gives the embedding context it otherwise lacks.

These are starting points, not settings. Optimal chunk size is a property of your documents and your queries — short factual lookups and long analytical questions want different values. The only reliable method is to vary it and measure retrieval recall on a golden set.

An embedding represents one idea. Put three unrelated topics in a chunk and the vector lands in the average of all three — near none of them. This is why oversized chunks quietly degrade retrieval even though nothing looks broken.

Chunking failures masquerade as model failures. When teams report "the LLM is hallucinating," the trace often shows the correct passage was split so badly it was never retrievable. Inspect the actual retrieved text before touching the model or the prompt.

5 · HYBRID RETRIEVAL

dense + sparse

	Dense (vector)	Sparse (BM25)
Matches	Meaning, paraphrase, synonyms	Exact tokens and their statistics
Strong on	"how do I get money back" → refund policy	SKUs, error codes, names, acronyms
Weak on	Rare literal strings, IDs, versions	Wording that differs from the document
Fails by	Confidently returning a near-miss	Returning nothing at all

Their failure modes are complementary, which is the whole point. Ask for "ERR_4021 handling" and dense search returns generic error-handling prose while BM25 finds the exact page. Ask "how do I stop the nightly job hanging" and BM25 finds nothing while dense search succeeds.

Reciprocal Rank Fusion — combining the two lists

```
# each document scored by its RANK in each list, not its score
score(d) = Σ  1 / (k + rank_i(d))
          i ∈ retrievers

# k is a smoothing constant; k = 60 is the widely used default
# from the original RRF paper (Cormack et al., 2009)
```

Why rank-based fusion rather than score-based. Cosine similarity and BM25 scores live on different, unbounded, query-dependent scales — normalising them is fragile and needs constant retuning. RRF ignores the scores entirely and uses only the ordering, so it works across retrievers without calibration. That robustness is why it is the common default.

Metadata filtering

Filtering by tenant, date, source or document type **before** the vector search both narrows the search space and enforces correctness. Check how your engine implements it: **pre-filtering** restricts the candidate set before traversal; **post-filtering** retrieves first and discards afterwards, which can return fewer than k results — or none — when the filter is selective.

6 · RERANKING

the highest-return addition

	Bi-encoder (retrieval)	Cross-encoder (rerank)
Encoding	Query and document encoded *separately*	Query and document encoded *together*
Precompute	Yes — documents embedded at ingest	No — must run per query-document pair
Cost	One embedding, then an index lookup	One model pass per candidate
Accuracy	Good	Substantially better
Scales to	Millions of documents	Tens of candidates

The reason the two-stage design exists. A cross-encoder is more accurate precisely because it lets query and document attend to each other — which is exactly why its result cannot be precomputed or indexed. You cannot run it over a million chunks. So you use the cheap model to find 100 plausible candidates and the expensive model to order them.

Where it fits

```
# retrieve wide, rerank narrow
candidates = hybrid_search(query, k=50..100)   # recall
ranked     = reranker(query, candidates)        # precision
context    = ranked[:3..10]                     # what the LLM sees
```

Candidate and final counts are STARTING POINTS — tune both against measured recall and your latency budget.

Reranker families

Family	Character
Hosted API rerankers e.g. Cohere Rerank, Jina	Strong quality, no infrastructure, per-call cost and an external hop
Open cross-encoders e.g. BGE-reranker	Self-hosted, no per-call fee, needs GPU capacity and ops
Late interaction e.g. ColBERT-style	Middle ground — partially precomputable, larger index

The cheapest experiment worth running. Adding a reranker to an existing pipeline touches one stage, requires no re-indexing, and is reversible. Measure recall and faithfulness before and after on the same golden set.

7 · VECTOR INDEX INTERNALS

what the knobs actually do

Index	How it works	Character
Flat brute force	Compares the query against every vector	Exact, 100% recall, linear cost. Correct choice for small corpora and as the accuracy baseline.
HNSW	Multi-layer proximity graph, greedily traversed	Fast and high recall; higher memory. The common default.
IVF	Vectors clustered; only nearby clusters are probed	Lower memory, needs training; recall depends on how many clusters are probed.

The parameters you will actually touch

Parameter	Effect
`M`	HNSW graph connectivity. Higher = better recall, more memory.
`efConstruction`	Build-time effort. Higher = better graph, slower indexing. Build cost only.
`efSearch`	**Query-time recall/latency dial.** Higher = better recall, slower queries. Tunable live.
`nlist / nprobe`	IVF: number of clusters, and how many are probed per query.

efSearch is the one to know. It trades recall against latency at query time without rebuilding the index — the fastest lever when retrieval quality is marginally short.

Quantization

Method	Trade
Scalar (int8)	Large memory reduction, modest recall loss
Product (PQ)	Much larger reduction, greater recall loss
Binary	Most aggressive; usually paired with a rescoring pass over full vectors

Approximate means approximate. ANN indexes trade recall for speed by construction — a passage that exists in your corpus can simply not be returned. Measure recall against a flat index on a sample before assuming retrieval is exhaustive.

Recall, latency, memory — pick two. Every index choice and every parameter on this panel is a position on that triangle. There is no configuration that wins all three.

8 · EVALUATION

the discipline that turns a demo into a system

Step 1 — build a golden set

A list of realistic questions, each paired with the **chunks that should be retrieved** and an acceptable answer. Sourced from real user queries and support tickets, not invented at a desk. Nothing else on this panel is possible without it.

```
# one golden record
{ "question": "refund window for damaged goods?",
  "relevant_chunk_ids": ["policy#returns-damaged"],
  "reference_answer": "14 days from receipt…",
  "tenant": "acme" }
```

Size it to be maintained, not to be impressive. A small set that is reviewed and kept current beats a large stale one. Grow it by adding every production failure you diagnose HEURISTIC .

Step 2 — measure retrieval on its own

Metric	Question it answers
Recall@k	Was the right chunk anywhere in the top k? **The first thing to measure.**
Precision@k	How much of what we retrieved was actually relevant?
MRR	How high up was the first correct result?
NDCG@k	Ranking quality with graded relevance and position discount

Recall@k bounds everything downstream. If the evidence is not retrieved, no reranker, prompt or model upgrade can recover it. Fix recall before tuning anything else.

Step 3 — measure generation separately

Metric	What it detects
Faithfulness groundedness	Is every claim supported by the retrieved context? **This is the hallucination metric.**
Answer relevancy	Does the answer actually address the question asked?
Context precision	Were the retrieved chunks useful, and ranked sensibly?
Context recall	Did retrieval capture everything the answer needed?

These four are the metric set popularised by the open-source **RAGAS** framework, and are widely used as a common vocabulary for RAG evaluation.

High faithfulness with low relevancy is a real and common state: the system faithfully summarises documents that do not answer the question. Both numbers are needed — either one alone can look healthy while the product is failing.

Step 4 — diagnose in a fixed order

#	Question	If no
1	Is the answer in the corpus at all?	Ingestion gap — not a RAG problem
2	Is it in a retrievable chunk?	Fix chunking
3	Is it in the top-k candidates?	Fix embeddings, hybrid, filters
4	Does it survive reranking?	Fix or add the reranker
5	Did the model use it faithfully?	Fix the prompt, ordering, or model

Skipping to step 5 is the classic mistake. Prompt engineering cannot repair a retrieval defect, but it will consume weeks of effort convincing you otherwise. Walk the steps in order, every time.

On LLM-as-judge. Automated graders make continuous evaluation affordable, but they are models with their own biases — including a documented tendency to favour longer answers. Calibrate the judge against human labels on a sample before trusting its verdicts.

9 · THE PATTERN LADDER

each rung answers a specific failure of the one below it — climb only when measurement says to

1 Simple RAG
`query → retrieve → LLM`
Use as: the baseline you measure everything else against.
Never ship it, but always build it first.

2 + Memory
`query + history → retrieve`
Fixes: follow-ups that only make sense in context.
Watch: rewrite the query before retrieving — raw pronouns retrieve nothing.
Minimum for anything conversational.

3 Branched
`decompose → parallel retrieve → merge`
Fixes: questions whose answer spans several documents.
Cost: N retrievals and an extra LLM call.
Comparisons and multi-part questions.

4 HyDE
`draft answer → embed that → search`
Fixes: question and answer embed differently, so the query is a poor probe.
Adds an LLM call to the critical path.

5 Adaptive
`route: none / simple / multi-step`
Fixes: paying to retrieve for queries that never needed it.
A cost and latency lever as much as a quality one.

6 Corrective
`retrieve → grade → retry / fallback`
Fixes: answering confidently from weak or irrelevant context.
A quality gate. Adds a retry tail to latency.

7 Self-RAG
`model critiques itself mid-answer`
Fixes: unsignalled low confidence in high-stakes answers.
Highest complexity. Justified in regulated domains.

8 Agentic
`orchestrator ↻ tools / sources`
Fixes: tasks a single fixed retrieval step cannot express.
Cost: unbounded loops unless you cap iterations.
Set a hard step limit and a timeout. Always.

9 Multimodal
`image/table → VLM → embed`
Fixes: answers that live in diagrams, charts and tables.
Most enterprise PDFs are not plain text.

10 Graph RAG
`entities + relationships → traverse`
Fixes: multi-hop questions where connection, not similarity, is the answer.
Cost: graph construction and maintenance is substantial.
Highest build cost. Wins where relationships are the question.

10 · LATENCY & COST

budget it per stage

Where the time goes

Stage	Relative contribution
Query embedding	Small — one short input
ANN + lexical search	Small to moderate; grows with `efSearch` and filters
Reranking	Moderate — scales with candidate count
LLM generation	**Usually dominant** — grows with input *and* output length

Optimise the dominant term. Shaving milliseconds off vector search while sending 20 loosely-relevant chunks to the model is optimising the wrong stage. Fewer, better chunks improve latency, cost and accuracy simultaneously — which is why reranking pays three times.

Perceived vs actual latency

- **Stream the answer** — time-to-first-token is what users experience
- Retrieval happens *before* the first token, so it is felt in full
- Show retrieval progress; a visible step feels far shorter than a blank screen

Cost structure

Component	Driver
Indexing (one-off)	Corpus size × embedding price
Re-indexing	Triggered by changing embedding model — a full migration
Query embedding	Query volume
Reranking	Volume × candidate count
LLM input tokens	**Usually the largest line** — retrieved context dominates

Precision is an economic argument, not only a quality one. Halving the context you send halves the dominant cost line and typically improves the answer.

Caching, with care. Exact-match caching is safe. Semantic caching returns a stored answer for a *similar* question — set the similarity threshold too loosely and it confidently serves answers to a question the user did not ask. Never cache across tenants or permission boundaries.

11 · SECURITY & TENANCY

where RAG leaks

Enforce permissions inside the retrieval filter. Attach ACL and tenant metadata to every chunk and apply it as a pre-filter. Retrieving broadly and discarding afterwards is not access control, and instructing the model not to reveal something is not a security boundary — it is a request.

Multi-tenant isolation

Approach	Trade-off
Index per tenant	Strongest isolation; overhead grows with tenant count
Namespace / partition	Good isolation with shared infrastructure — common default
Shared index + filter	Cheapest and most dangerous: one missing filter leaks across tenants

If you use a shared index, make the tenant filter impossible to omit — inject it in a wrapper that no caller can bypass, and write a test that fails when it is missing.

Indirect prompt injection

Retrieved text enters the prompt. If an attacker can influence any indexed document — a shared wiki page, an uploaded file, a support ticket — they can plant instructions that the model may follow. **Retrieved content is untrusted input.**

- Delimit retrieved text clearly and instruct the model to treat it as **data, not instructions**
- Constrain what the surrounding system is permitted to do, especially with tools
- Validate outputs — particularly anything triggering an action
- Control who can write into the corpus, and log it

Embeddings are sensitive data. A vector is a lossy but informative encoding of its source text; published research on embedding inversion has shown meaningful portions of original text can be reconstructed from embeddings alone. Treat the vector store with the same controls as the source documents — encryption, access control, retention, and deletion.

Deletion must propagate. A right-to-erasure request has to remove the source document, its chunks, its vectors, and any cached answers derived from them.

12 · RUNNING IT

freshness and observability

Keeping the index current

Event	Required handling
New document	Chunk, embed, upsert — the easy case
Updated document	Replace *all* prior chunks for that ID, or stale ones linger alongside new
Deleted document	Remove chunks and vectors. Most commonly missed.
Permission change	Update ACL metadata — otherwise retrieval enforces yesterday's rules
Model change	Re-embed the whole corpus; plan as a migration with a dual-index cutover

Version your index. Pin the embedding model and chunking config used to build it — without that record, you cannot tell whether a regression came from your data or your configuration.

What to log on every query

- The query, and the rewritten query if it was rewritten
- Candidates from each retriever, with scores and ranks
- The reranked set, and the chunk IDs actually sent to the model
- The answer, its citations, and per-stage latency
- User feedback signals — explicit ratings and implicit behaviour

Log the retrieved chunk IDs above all else. Almost every quality investigation starts with the question "what did the model actually see?" A system that cannot answer that can only be debugged by guesswork.

Monitor for drift

Signal	Suggests
Retrieval scores trending down	Queries drifting away from corpus coverage
Rising "no relevant results"	A content gap users are hitting
Faithfulness falling	Retrieval degradation, or a model change upstream

Failed queries are your roadmap. Questions that retrieve nothing tell you what content to write next — and each belongs in the golden set.

13 · PRODUCTION READINESS CHECKLIST

what to confirm before this carries real traffic

- ☐ **Golden set exists** — real questions with known-correct source chunks, kept current
- ☐ **Retrieval measured separately** — Recall@k tracked and known, not assumed
- ☐ **Generation measured separately** — faithfulness and relevancy both tracked
- ☐ **Recall verified against a flat index** — you know what ANN is missing
- ☐ **Hybrid retrieval in place** — exact IDs and codes are findable
- ☐ **Reranking evaluated** — measured before and after on the same set
- ☐ **Chunking tuned on your data** — not copied from a tutorial
- ☐ **Parent expansion or overlap** — boundary loss is mitigated
- ☐ **Embedding model pinned and recorded** — with a re-embedding migration plan
- ☐ **ACL enforced as a pre-filter** — never post-filtered, never left to the prompt
- ☐ **Tenant isolation cannot be bypassed** — with a test that fails if the filter is dropped
- ☐ **Retrieved content treated as untrusted** — injection considered and mitigated
- ☐ **Vector store secured like source data** — encryption, access control, retention
- ☐ **Delete path works end to end** — source, chunks, vectors, caches
- ☐ **Incremental indexing handles updates** — no stale duplicate chunks
- ☐ **Latency budgeted per stage** — and the dominant term identified
- ☐ **Cost per query modelled** — input tokens accounted for
- ☐ **Answers stream** — time-to-first-token optimised
- ☐ **Every query traced** — retrieved chunk IDs logged
- ☐ **Citations returned** — users can verify claims against source
- ☐ **Refusal path exists** — the system can say it does not know
- ☐ **Agentic loops bounded** — hard step limits and timeouts
- ☐ **Semantic cache thresholds validated** — and never shared across tenants
- ☐ **Drift monitored** — scores, no-result rate, faithfulness, p95 latency
- ☐ **Failed queries feed back** — into the golden set and the content roadmap
- ☐ **Rollback possible** — index versioned, previous configuration restorable

The single most common root cause. Across systems that underperform in production, the defect is far more often in retrieval — chunking, recall, ranking — than in the language model. Teams reach for a bigger model because it is the easiest thing to change, not because it is what the evidence points to. Measure retrieval first; it is usually where the answer is.